Coinbase Account Policies

Security Review

Cantina Managed review by:
Chris Smith, Lead Security Researcher
Chinmay Farkya, Security Researcher

March 31, 2026

Contents

1 Introduction **2**
1.1 About Cantina . . . 2
1.2 Disclaimer . . . 2
1.3 Risk assessment . . . 2
1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**
2.1 Scope . . . 3

3 Findings **4**
3.1 Low Risk . . . 4
3.1.1 `onExecute` should not block `installWithSignature` . . . 4
3.1.2 Changing `PolicyManager` breaks several Policy Manager → Policy assumptions . . . 4
3.1.3 In `_replace()` logic, the same replaceData is being passed to old and new policies . . . 4
3.1.4 `PolicyManager` doesn't allow installing a policy with validity window in future . . . 5
3.2 Gas Optimization . . . 6
3.2.1 Re-accessed storage for `installOnReplace` and `uninstallOnReplace` . . . 6
3.3 Informational . . . 6
3.3.1 Separate Admin and Pauser roles . . . 6
3.3.2 Old repository reference for `PublicERC6492Validator` . . . 6
3.3.3 Wasted storage read in `replaceWithSignature` . . . 6
3.3.4 Set uninstalled earlier and unconditionally . . . 7
3.3.5 Incorrect Natspec . . . 7
3.3.6 `address.code.length` checks are not useful post ERC7702 . . . 8
3.3.7 Executors can spam `NonceCancelled` events on `AOAPolicy` even after a policyID has been uninstalled . . . 8
3.3.8 `PolicyManager` system is not compatible with ERC-7579 style Modular Smart Wallets 9
3.3.9 Lack of validation for Morpho vaults in `MorphoLendPolicy` . . . 9
3.3.10 Passing `effectiveCaller` as `msg.sender` potentially confusing for Policies . . . 10

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	**Impact: High**	**Impact: Medium**	**Impact: Low**
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Base is a secure, low-cost, builder-friendly Ethereum L2 built to bring the next billion users onchain.

From Mar 1st to Mar 10th the Cantina team conducted a review of account-policies on commit hash 540dc78d. The team identified a total of **15** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	4	3	1
Gas Optimizations	1	1	0
Informational	10	7	3
Total	**15**	**11**	**4**

2.1 Scope

The security review had the following components in scope for account-policies on commit hash 540dc78d:

```
src
├── PolicyManager.sol
├── PublicERC6492Validator.sol
├── interfaces
│   └── morpho
│       ├── BlueTypes.sol
│       ├── IMorphoBlue.sol
│       ├── IMorphoVault.sol
│       └── IOracle.sol
└── policies
    ├── AOAPolicy.sol
    ├── MorphoLendPolicy.sol
    ├── MorphoLoanProtectionPolicy.sol
    ├── Policy.sol
    └── accounting
        └── RecurringAllowance.sol
```

3 Findings

3.1 Low Risk

3.1.1 `onExecute` should not block `installWithSignature`

Severity: Low Risk

Context: AOAPolicy.sol#L336-L343

Description: `install` and `installWithSignature` currently have different behavior when a policy is paused. Because `installWithSignature` always calls `AOAPolicy._onExecute` it will revert if a policy is paused. This creates a different install path for users acting directly versus ones relying on signatures and may create a disincentive for a policy owner/pauser to pause a policy in order to keep allowing installs.

Recommendation: Allowing install without execution in the `installWithSignature` path would align the two paths. This can be accomplished by allowing the `installWithSignature` path to skip execution by not passing `executionData` and removing the modifier on the function relying on the internal function instead:

```
function _onExecute(
        bytes32 policyId,
        address account,
        bytes calldata policyConfig,
        bytes calldata executionData,
        address caller
    ) internal override returns (bytes memory accountCallData, bytes memory
    ↪ postCallData) {
        if (executionData.length == 0) return (accountCallData, postCallData);
        _requireNotPaused();
//...
```

Coinbase: Accepted and fix implemented in PR 23.

Cantina Managed: Fix verified.

3.1.2 Changing `PolicyManager` breaks several Policy Manager → Policy assumptions

Severity: Low Risk

Context: AOAPolicy.sol#L143-L153

Description: Currently, policies do not clear out the config hash on uninstall. This should be fine as once uninstalled, a policy cannot be reinstalled on the Policy Manager. However, allowing policies to change managers opens the potential of reusing uninstalled config hashes and reinstalling a previously uninstalled policy (the same `policyId` installed on the second PolicyManager would look like a fresh install from that policy manager even if it had been uninstalled by a different policy manager).

Recommendation: There is not a great use-case for allowing a policy to change policy managers as the second policy manager would have to behave very differently or would require an install on the policy even though state inside the policy might already be installed. It would be better to enforce that policies are tied to their manager and that moving to a new PolicyManager requires a user call install (or allow installWithSignature) on the new PolicyManager.

Coinbase: Acknowledged, won't fix but added documentation calling out the broken assumptions.

Cantina Managed: Acknowledged.

3.1.3 In `_replace()` logic, the same replaceData is being passed to old and new policies

Severity: Low Risk

Context: PolicyManager.sol#L769-L773

Description: The `_replace()` flow calls `_uninstallForReplace()` on the old policy contract, and `_installForReplace()` on the new policy contract.

```
_uninstallForReplace(
    oldPolicy, oldPolicyId, oldPolicyConfig, replaceData, newBinding.policy,
    ↪  newPolicyId, msg.sender
);

_installForReplace(newBinding, replaceData, oldPolicy, oldPolicyId);
```

We can see that this logic forwards the same `replaceData` to both the policy contracts.

Here is the implementation for `_uninstallForReplace()` function and for `_installForReplace()` function.

The logic in both these functions calls `policy.onReplace()` but this can be an issue in cross-policy replacement because the same replaceData is being passed down to both policy contracts.

Both policies are allowed to be different contracts, with different needs for the value of "replaceData".

In the current AOAPolicy, this is not an issue but this will make replace unusable whenever the required replaceData is incompatible between the outgoing policy and incoming policy.

Recommendation: Consider modifying replace logic to accept two different replaceData values as input, one each for the old policy and the new policy.

Coinbase: Fixed by modifying `ReplacePayload.replaceData` into two independent fields: `oldPolicyReplaceData` and `newPolicyReplaceData` in PR 24.

Cantina Managed: Fix verified.

3.1.4 `PolicyManager` doesn't allow installing a policy with validity window in future

Severity: Low Risk

Context: PolicyManager.sol#L562

Description: In installation flows, the current timestamp is checked against the validity window using `_checkValidityWindow()`:

```
function _checkValidityWindow(uint40 validAfter, uint40 validUntil) internal view {
    uint40 currentTimestamp = uint40(block.timestamp);
    if (validAfter != 0 && currentTimestamp < validAfter) revert
    ↪  BeforeValidAfter(currentTimestamp, validAfter);
    if (validUntil != 0 && currentTimestamp >= validUntil) revert
    ↪  AfterValidUntil(currentTimestamp, validUntil);
}
```

In this function, `validAfter` is not allowed to be in the future, and it requires the installation window as well as execution window to be the same.

This limits the use cases of policies designed for PolicyManager, especially because some type of subscription services cannot be designed like this.

Pre-installing a policy should be allowed, before the actual execution window is supposed to start.

- If a service wants to offer a free trial and start a subscription in near future, with payments allowed only after a certain timestamp.
- Then at day 1, user should be able to install that policy with an approval to deduct payments starting from day 7.

Recommendation: Consider allowing `validAfter` to be in future, only for the installation flows (i.e. `_install()` and `_installForReplace()`). Note that current `installWithSignature()` flow will not work if `validAfter` is allowed to be in future, since that includes execution which is (and should be) blocked prior to the validity window.

To mitigate this, the `_checkValidityWindow()` call in `_execute()` should be deferred to only be called after the `accountCallData` is received and is non-empty.

Coinbase: Accepted and fix implemented in PR 24.

Cantina Managed: Fix verified.

3.2 Gas Optimization

3.2.1 Re-accessed storage for `installOnReplace` and `uninstallOnReplace`

Severity: Gas Optimization

Context: PolicyManager.sol#L751-L752, PolicyManager.sol#L769-L771, PolicyManager.sol#L801-L802, PolicyManager.sol#L838-L840

Description: `installOnReplace` is only ever called from `_replace` which already has `newPolicyRecord` loaded. It could be passed to `_installOnReplace` to avoid regeneration of the `policyId` and re-`SLOAD` of the `policyRecord`. The same applies to `_uninstallOnReplace`.

Coinbase: Accepted and fix implemented in PR 23.

Cantina Managed: Fix verified.

3.3 Informational

3.3.1 Separate Admin and Pauser roles

Severity: Informational

Context: AOAPolicy.sol#L158-L167, MorphoLendPolicy.sol#L74

Description: Consider separating the ability to pause the contract (something you might want to do in a more rapid response situation) and admin the contract (something that should have more security around the keys).

Coinbase: Accepted, fix implemented in PR 22.

Cantina Managed: Fix Verified.

3.3.2 Old repository reference for `PublicERC6492Validator`

Severity: Informational

Context: PublicERC6492Validator.sol#L13

Finding Description: The reference for the `PublicERC6492Validator` is for a different repository, but should probably point to this one.

Coinbase: Accepted and fix implemented in PR 23.

Cantina Managed: Fix verfied.

3.3.3 Wasted storage read in `replaceWithSignature`

Severity: Informational

Context: PolicyManager.sol#L376-L383, PolicyManager.sol#L401-L405, PolicyManager.sol#L751-L761

Description: These lines in `replaceWithSignature` are redundant with the check in `replace` which early returns the `newPolicyId`. The control flow seems to be the same with or without these lines.

Recommendation: Remove these and remove the conditional around `_replace` allowing it to return early if the policy replace action has already occurred.

Coinbase: Accepted and fix implemented in PR 23.

Cantina Managed: Fix verified.

3.3.4 Set uninstalled earlier and unconditionally

Severity: Informational

Context: PolicyManager.sol#L612-L638

Description: In both conditions (`policyRecordByBinding.installed == true|false`), `policyRecordByBinding.uninstalled` is set to `true`. It would be simpler to simply set it as soon as possible in the flow.

Recommendation:

```
if (policyRecordByBinding.uninstalled) return policyId;
policyRecordByBinding.uninstalled = true;

// Installed lifecycle: uninstall by policyId.
if (policyRecordByBinding.installed) {
    try Policy(binding.policy)
        .onUninstall(
            policyId,
            policyRecordByBinding.account,
            binding.policyConfig,
            payload.uninstallData,
            effectiveCaller
        ) {}
    catch {
        // If the hook reverts and the effective caller is not the account, revert
        if (effectiveCaller != policyRecordByBinding.account) {
            revert Unauthorized(effectiveCaller);
        }
    }
    emit PolicyUninstalled(policyId, policyRecordByBinding.account, binding.policy);
    return policyId;
}

// Pre-install uninstallation: config must be non-empty in the binding.
if (binding.policyConfig.length == 0) revert InvalidPayload();
```

Coinbase: Accepted and fix implemented in PR 23.

Cantina Managed: Fix verified.

3.3.5 Incorrect Natspec

Severity: Informational

Context: AOAPolicy.sol#L123-L124, Policy.sol#L223, PolicyManager.sol#L181-L182

Description: There are a few instances of incorrect natspec comments:

- `PolicyNotInstalled()` error: `Thrown when attempting to install a policyId that is already installed.` should be changed to `Thrown when attempting to replace with a policyId that is already installed`.
- `NonceCancelled` event: `@param canceller The address that cancelled the nonce (executor or account).` should be changed to `@param canceller The address that cancelled the nonce (executor)`.
- `_onUninstall()` function: `Revert to refuse non-account uninstallation` should be changed to `Revert to refuse unauthorized uninstallation`.

Recommendation: Consider modifying the comments as mentioned.

Coinbase: Accepting suggestion. Fix is in PR 24.

Cantina Managed: Fix verified.

3.3.6 `address.code.length` checks are not useful post ERC7702

Severity: Informational

Context: AOAPolicy.sol#L149, MorphoLendPolicy.sol#L127, MorphoLoanProtectionPolicy.sol#L115, Policy.sol#L68, PolicyManager.sol#L241

Description: At several places in the codebase, the protocol is trying to ensure that certain addresses are smart contracts and not EOAs, by checking code.length for the given address. For example:

```
if (address(publicERC6492Validator).code.length == 0) {
    revert ValidatorNotContract(address(publicERC6492Validator));
```

However, these checks are not useful after EIP-7702 because EIP-7702 fundamentally breaks the long-standing assumption that `address.code.length == 0` (or `extcodesize` returning 0) guarantees an address is an Externally Owned Account (EOA). With EIP-7702, an EOA can temporarily possess bytecode, making it appear as a smart contract to on-chain checks.

There are 5 instances of this issue:

- PolicyManager.sol#L241.
- AOAPolicy.sol#L149.
- Policy.sol#L68.
- MorphoLendPolicy.sol#L127.
- MorphoLoanProtectionPolicy.sol#L115.

Recommendation: In order to ensure that a given address is not an EOA, we need to use a check like this:

```
bytes3 internal constant _DELEGATION_PREFIX = 0xef0100;
function isEOA(address target) internal view returns (bool) {
    bytes memory code = target.code;
    return (code.length == 0 || bytes3(code) == _DELEGATION_PREFIX);
}
```

Coinbase: Accepting suggestion. Fix is in PR 24.

Cantina Managed: Fix verified.

3.3.7 Executors can spam `NonceCancelled` events on `AOAPolicy` even after a policyID has been uninstalled

Severity: Informational

Context: AOAPolicy.sol#L181-L195

Description: In `AOAPolicy.sol`, even after a policy has been uninstalled as per the `PolicyManager`, the executor can keep calling `cancelNonces()` for that `policyId` and this will keep emitting `NonceCancelled` events.

```
function cancelNonces(bytes32 policyId, uint256[] calldata nonces, bytes calldata
↪  policyConfig) external {
    _requireConfigHash(policyId, policyConfig);
    (AOAConfig memory aoaConfig,) = _decodeAOAConfig(policyConfig);

    if (msg.sender != aoaConfig.executor) {
        revert UnauthorizedCanceller(msg.sender, aoaConfig.executor);
    }

    for (uint256 i; i < nonces.length; ++i) {
        if (!_usedNonces[policyId][nonces[i]]) {
            _usedNonces[policyId][nonces[i]] = true;
            emit NonceCancelled(policyId, nonces[i], msg.sender);
        }
    }
}
```

The `configHash` is never cleared even when a `policyId` is uninstalled, so there is nothing to stop the executor. They can keep running these calls which will obfuscate event data for the policies that inherit from `AOAPolicy.sol`.

Recommendation: The `cancelNonces()` function should revert if a `policyId` has been uninstalled.

Coinbase: Acknowledged. The event spam is also possible in an installed state, and holds no other risk. Besides, it is also expensive for the executors.

Cantina Managed: Acknowledged.

3.3.8 `PolicyManager` system is not compatible with ERC-7579 style Modular Smart Wallets

Severity: Informational

Context: PolicyManager.sol#L704-L709

Description: The goal of the `PolicyManager` is to offer a wallet-agnostic permission system, but it is not compatible with most wallets that work as Modular smart wallets. It only seems to be compatible with wallet designs similar to Coinbase smart wallet.

The `_execute()` function in `PolicyManager` calls user's smart wallet with the `accountCallData` received from the policy.

Thus, the user wallet needs to have an "execute..." function which is callable externally, just like the `executeBatch` function in Coinbase smart wallet.

The Coinbase implementation has an owner system where `PolicyManager` can be added as an owner, so that it call the functions guarded by `onlyEntrypointOrOwner` : like executeBatch() function.

But in Modular Smart Wallets like Biconomy, Zerodev Kernel etc.,there is no such execute/ executeBatch function (all their execute functions are guarded by onlyEntrypoint, so they cannot be called directly by `PolicyManager`).

There is only one potentially useful function `executeFromExecutor()`, and that can only be called by a ERC-7579 style Executor Module, but `PolicyManager` is not structured to mimic the interface of an ERC-7579 Executor Module.

An executor module needs to have `onInstall()` and `onUninstall()` functions to work with the user wallet.

Any calls from PolicyManager => one of these Modular smart wallet will always fail, meaning the PolicyManager is not compatible with them.

Recommendation: Consider documenting which wallets/wallet designs are meant to be supported by the PolicyManager, and clearly define in what ways it operates as wallet-agnostic.

Coinbase: Acknowledged. We might consider this in future. We evaluated the feasibility of a 7579 adapter and concluded it is viable: a 7579 executor module could receive calls from the `PolicyManager` and translate them into `executeFromExecutor` calls on the account. This would be backward-compatible with the protocol as currently written. This is a future expansion path.

Cantina Managed: Acknowledged.

3.3.9 Lack of validation for Morpho vaults in `MorphoLendPolicy`

Severity: Informational

Context: MorphoLendPolicy.sol#L125-L130

Description: The installation flows in `MorphoLendPolicy` lead to `_onAOAInstall()` function, where the following validation is performed on the input params.

```
function _onAOAInstall(bytes32, address, AOAConfig memory, bytes memory
↪  policySpecificConfig) internal override {
    LendPolicyConfig memory lendPolicyConfig = abi.decode(policySpecificConfig,
    ↪  (LendPolicyConfig));
    if (lendPolicyConfig.vault.code.length == 0) revert
    ↪  VaultNotContract(lendPolicyConfig.vault);
```

```
    if (lendPolicyConfig.depositLimit.period == 0) revert
    ↪  RecurringAllowance.ZeroPeriod();
    if (lendPolicyConfig.depositLimit.allowance == 0) revert
    ↪  RecurringAllowance.ZeroAllowance();
}
```

But nowhere in this logic is the `vault` address verified to be an actual Morpho vault.

The user wallet installing the policy controls the `policyConfig`, and hence the "vault" address encoded in this `policyConfig`. This vault is not guaranteed to be a Morpho vault, and if some other smart contract is used in this place, it could lead to unintended consequences.

Recommendation: Consider adding another check to `_onAOAInstall()` which validates that the `vault` is a legit Morpho vault. This can be done by fetching vault details from Morpho's official vault factory contracts.

Coinbase: Acknowledged, won't fix. Overhead of managing factory config likely outweighs benefit operationally.

Cantina Managed: Acknowledged.

3.3.10 Passing `effectiveCaller` as `msg.sender` potentially confusing for Policies

Severity: Informational

Context: PolicyManager.sol#L770

Description: After discussing with client, it is more clear that passing only the account address and not the sending address in the case of `installWithSignature` and `replaceWithSignature` is a more clear pattern. The `PolicyManager` will be responsible for validating the intent of the signer on that paths, then once that is confirmed the policy should treat their validations the as coming from the account directly.

Coinbase: We will choose not to provide information about `msg.sender` to policies for any of the install/replacement-related hooks, dissuading them from implementing logic based on this value that should be irrelevant to replacement auth. `replaceData` will serve as the vector for any extra-account authorization or behavior that may need to be provided for replacement actions.

Cantina Managed: Fix verified for in-scope contracts (changes to `src/policies/SingleExecutorAuthorizedPolicy.sol` in PR 25 were not verified as that policy was not in scope.

Cantina Managed: Fix verified for in-scope contracts.